CSD Lab 7: High-Performance Spatial Proximity & Graph Traversal Recommendation API

Name: Venkat | **Student ID:** 12341070
Student System: `stu9_sys4` | **Host IP:** `10.1.75.51` | **API Port:** `3236`
GitHub Repository: https://github.com/Venkat-1905/csdlab7

October 9, 2026

Abstract

This report documents the design, algorithmic formulation, implementation, and deployment of a high-performance RESTful spatial recommendation API. Given a dataset of 10,000 geographical locations positioned on a discrete 1.0×1.0 coordinate lattice, the objective is to recommend the 10 closest valid locations matching a requested category and search radius. Crucially, while eligibility is defined by straight-line Euclidean circular proximity, true spatial closeness is governed by actual road network traversal distance across a dynamically provided linkage graph. We describe our $\mathcal{O}(1)$ lattice coordinate mapping, inverted category indexing, squared Euclidean pruning, cache-conscious array-backed graph construction, and min-heap Dijkstra shortest path computation. Finally, we document the production deployment on remote container `stu9_sys4` with host port forwarding to port `3236` and a high-availability watchdog supervisor.

1 Introduction and Problem Specification

In conventional spatial information systems, proximity is frequently approximated via straight-line Euclidean or Great-Circle distance. However, in physical road networks and constrained urban environments, obstacles and missing links mean that straight-line proximity often diverges significantly from actual traversable distance.

The objective of this assignment is to construct a production-ready REST API satisfying the following specifications:

1. **Dataset**: A CSV file containing 10,000 spatial records formatted as `[ID, Latitude, Longitude, Category]`. The points reside on a unit grid $[0.0, 1.0] \times [0.0, 1.0]$.
2. **API Endpoint**: The service must be exposed at the route `/search/` accepting five parameters:
   - `lat`: Query Latitude $\in [0.0, 1.0]$.
   - `long`: Query Longitude $\in [0.0, 1.0]$.
   - `cat`: Category filter (e.g., `bank`, `hospital`, `store`, etc.).
   - `rad`: Circular search radius r.
   - `link`: Dynamic text file containing space-separated undirected road segments $u \leftrightarrow v$.
3. **Output**: Exactly 10 recommended location IDs representing the closest valid candidates by shortest network traversal distance.
4. **Distance Semantics**:

- *Radius Constraint*: A location p is valid if and only if $\text{EuclideanDist}(q, p) \leq r$.
- *Closeness Measure*: Candidate ranking is computed by shortest path network distance $d_G(q, p)$ along edges provided in `link`.

2 System Architecture and Algorithms

2.1 Lattice Structure Analysis and $\mathcal{O}(1)$ Snapping

Exploratory data analysis of `locations.csv` reveals that the 10,000 locations are arranged as an exact 100×100 regular 2D lattice spanning $[0.0, 1.0]$. The coordinate step size between adjacent lattice coordinates is:

$$\Delta = \frac{1.0}{99} \approx 0.01010101\ldots \tag{1}$$

The unique location identifier maps directly to 2D matrix coordinates via:

$$\text{ID}(i, j) = i \times 100 + j + 1, \quad i, j \in \{0, 1, \ldots, 99\} \tag{2}$$

where i represents the row (latitude index) and j represents the column (longitude index).

When an arbitrary floating-point coordinate (lat_q, lon_q) is submitted, the corresponding source grid node s is identified in $\mathcal{O}(1)$ time through uniform rounding:

$$i = \text{clip}\left(\left\lfloor \frac{lat_q}{\Delta} + 0.5 \right\rfloor, 0, 99\right), \quad j = \text{clip}\left(\left\lfloor \frac{lon_q}{\Delta} + 0.5 \right\rfloor, 0, 99\right) \tag{3}$$

This replaces expensive spatial KD-Tree searches with an exact constant-time algebraic calculation.

2.2 Category Inverted Index and Euclidean Pruning

The dataset is uniformly partitioned across eight distinct categories (`bank`, `store`, `pharmacy`, `hospital`, `school`, `restaurant`, `park`, `cafe`), with exactly 1,250 entities per category.

To optimize query execution:

1. **Inverted Category Index**: An in-memory inverted table $\mathcal{I} : \text{Category} \rightarrow \text{List[ID]}$ partitions the candidate pool, immediately eliminating 87.5% of irrelevant entries without scanning.

2. **Squared Euclidean Radius Filter**: To filter locations within circular radius r, we test:

$$(lat_k - lat_q)^2 + (lon_k - lon_q)^2 \leq r^2 \tag{4}$$

   By comparing squared values directly, costly floating-point square root ($\sqrt{\cdot}$) operations are completely avoided during the candidate selection stage.

2.3 Dynamic Graph Representation

Because road linkages are submitted dynamically per request via the `link` upload, graph construction must be exceptionally fast:

- We allocate a 1-indexed contiguous array of adjacency lists of size 10,001.
- Each line in the uploaded file contains two whitespace-separated integers u and v.
- Edge weights correspond to the Euclidean distance between the lattice vertices:

$$w(u, v) = \sqrt{(lat_u - lat_v)^2 + (lon_u - lon_v)^2} \tag{5}$$

- Since edges are undirected, both (v, w) and (u, w) are appended to the respective adjacency lists.

2.4 Shortest Path Traversal: Dijkstra's Algorithm

To compute the true road network distance from snapped source s to all target candidates:

1. An array distance vector $\mathbf{d} \in \mathbb{R}^{10001}$ is initialized with $\mathbf{d}[s] = 0$ and $\mathbf{d}[v] = \infty$ for all $v \neq s$.
2. A min-heap priority queue storing pairs (d, u) is initialized with $(0, s)$.
3. In each iteration, vertex u with minimal distance is popped. If $d > \mathbf{d}[u]$, it is discarded as stale. Otherwise, each incident edge (v, w) is relaxed:

$$\text{if } \mathbf{d}[u] + w < \mathbf{d}[v] \implies \mathbf{d}[v] \leftarrow \mathbf{d}[u] + w, \quad \text{push } (\mathbf{d}[v], v) \tag{6}$$

2.5 Candidate Ranking and Output Formatting

After computing traversal distances:

1. Valid candidate locations within the circular radius that are reachable ($\mathbf{d}[v] < \infty$) are collected.
2. The list is sorted primarily by graph distance $\mathbf{d}[v]$ in ascending order, with ID v acting as a deterministic tie-breaker:

$$\text{key}(v) = (\mathbf{d}[v], v) \tag{7}$$

3. The first 10 location IDs are sliced and serialized into the required JSON structure:

```
{
  "recommended_locations": [id_1, id_2, ..., id_10]
}

```

3 Complexity Analysis

Table 1: Algorithmic Complexity Summary

Pipeline Stage	**Data Structure / Method**	**Time Complexity**	**Space Complexity**
Data Preloading	Inverted Hash Map	$\mathcal{O}(N)$	$\mathcal{O}(N)$
Lattice Snapping	Algebraic Rounding	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Candidate Filtering	Category Index + Squared Dist	$\mathcal{O}(N_{cat}) \approx 1,250$ ops	$\mathcal{O}(N_{cat})$
Graph Construction	Array Adjacency List	$\mathcal{O}(\lvert E\rvert) \approx 17,000$ edges	$\mathcal{O}(\lvert V\rvert + \lvert E\rvert)$
Graph Traversal	Min-Heap Dijkstra	$\mathcal{O}((\lvert V\rvert + \lvert E\rvert) \log \lvert V\rvert)$	$\mathcal{O}(\lvert V\rvert)$
Top-10 Sorting	TimSort / QuickSort	$\mathcal{O}(K \log K), K \leq 1,250$	$\mathcal{O}(K)$
Total Query Cost		$\approx$ **50 – 75 ms**	$\approx$ **35 MB RAM**

4 System Deployment and Port Forwarding

4.1 Remote Server Infrastructure

The service is deployed on the designated institutional compute server:

- **Gateway Host**: `10.1.75.51`
- **Container Environment**: `stu9_sys4`
- **SSH Administration**: Accessible via `ssh -p 2236 student@10.1.75.51`

4.2 Port Mapping Strategy

Available ports in the range $[3000, 7000]$ were systematically probed:

- Port `5000` was occupied by an existing system service (`backend_server`).
- Port `3000` was selected and allocated for our application.
- Based on the container's NAT port forwarding rules (where container SSH port `22` maps to host port `2236`), container port `3000` maps directly to host port `3236`.

Hence, the globally accessible endpoint is:

$$\textbf{http} : //\textbf{10.1.75.51} : \textbf{3236}/\textbf{search}/$$

4.3 High-Availability Process Watchdog

To ensure continuous operation without manual intervention, a dedicated supervisor daemon script (`run_server.sh`) was deployed:

```
#!/bin/bash
cd /home/student/csdlab7
while true; do
    echo "[$(date)] Starting Flask app.py on port 3000..." >> runner.log
    python3 /home/student/csdlab7/app.py >> server.log 2>&1
    sleep 2
done
```

The script was executed in a detached subshell (`nohup ... & disown`), ensuring that closing the SSH session or terminal has zero impact on server availability.

4.4 Source Code Version Control

The complete codebase, deployment automation scripts, test harnesses, and documentation are tracked using Git and hosted on GitHub:

Repository URL: https://github.com/Venkat-1905/csdlab7

5 Experimental Verification and Results

The API was rigorously tested across multiple query configurations from both the local development workstation and external HTTP clients (cURL and Postman).

5.1 Test Case 1: Central Coordinate Query

- **Parameters**: `lat=0.5`, `long=0.5`, `cat=bank`, `rad=0.15`
- **cURL Request**:

```
curl -X POST http://10.1.75.51:3236/search/ \
  -F "lat=0.5" -F "long=0.5" -F "cat=bank" -F "rad=0.15" \
  -F "link=@test_links.txt"
```

- **Result (200 OK, 75ms)**:

```
{
  "recommended_locations": [4850, 4948, 5155, 5452, 5551, 4753, 5349, 4947,
    5651, 5156]
}
```

5.2 Test Case 2: Corner Coordinate Query

- **Parameters**: `lat=0.9`, `long=0.9`, `cat=hospital`, `rad=0.25`
- **Result (200 OK, 64ms)**:

```
{
  "recommended_locations": [8890, 8788, 8688, 8690, 8793, 8693, 8488, 8785,
    8593, 8984]
}
```

5.3 Live Verification Log

Live inspection of `/home/student/csdlab7/server.log` confirms successful client interaction:

```
 * Running on http://127.0.0.1:3000
 * Running on http://172.17.0.37:3000
10.50.8.51 - - [09/Oct/2026 16:52:15] "GET / HTTP/1.1" 200 -
10.50.8.51 - - [09/Oct/2026 16:52:15] "POST /search/ HTTP/1.1" 200 -
10.50.8.51 - - [09/Oct/2026 16:55:49] "POST /search/ HTTP/1.1" 200 -
```

6 Conclusion

The spatial recommendation API successfully achieves all functional, algorithmic, and operational objectives:

- Accurately evaluates circular Euclidean radius constraints while resolving true network closeness via min-heap Dijkstra traversal.
- Optimizes throughput with $\mathcal{O}(1)$ lattice snapping, inverted category indexing, and squared-distance comparisons, delivering sub-80ms response times for 10,000-node lattices.
- Successfully deployed and active on remote container `stu9_sys4` via forwarded host port `3236` under continuous watchdog supervision.